

Extensions

You can use `pinionai_extensions.py` page to add your own enterprise or python functions to `pinionai` and extend your own internal functions to be used in the `PinionAIClient`.

pinionai_extensions.py

Developers can add functions here. These can be used by creating functional declarations in the PinionAI Studio's Tools page. So enterprises can use specialized code and other resources by extending PinionAI to their own infrastructure, make calls or perform other actions.

Create a Function, Call it with a Tool

Example: Stock Data Custom Function

Step 1: Add a Python Function

Create and add an async function to be defined in `/pinionai_extensions.py`. The LLM will use the arguments from the user's request to call this function.

```
# Stock Market Tool
async def get_stock_data(
    stock_lookup_function: str | None = None,
    stock_symbol: str | None = None,
    alphavantage_key: str | None = None,
    interval: str | None = None,
) -> dict:
    """
    Fetches stock data from the Alpha Vantage API.
    """
    # Filter out None values from parameters
    params = {
        "function": stock_lookup_function,
        "symbol": stock_symbol,
        "apikey": alphavantage_key,
        "interval": interval,
    }
    params = {k: v for k, v in params.items() if v is not None}
    try:
        async with httpx.AsyncClient() as client:
            base_url = 'https://www.alphavantage.co/query'
            response = await client.get(base_url, params=params, headers=
{"User-Agent": "none"})
            logging.debug(f"Stock check Response URL: {response.url}")
            response.raise_for_status()
            # convert to markdown
            stock_data = response.json()
            return await format_stock_data_as_markdown(stock_data)
    except httpx.HTTPStatusError as http_err:
```

```

        logging.error(f"HTTP error occurred: {http_err} -
{http_err.response.text}")
        return {"error": f"HTTP error: {http_err.response.status_code}",
"message": http_err.response.text}
    except Exception as e:
        logging.error(f"An unexpected error occurred: {e}")
        return {"error": "An unexpected error occurred.", "message":
str(e)}

```

Step 2: Create a new tool to call it:

You can define your own custom tools by providing a function schema in the Functional Declaration JSON field for a tool. This tells the LLM what your function does and what parameters it needs.

Important: For every Python function, you create a corresponding Tool with a functional declaration here with the exact same name of the function inside the `pinionai_extensions.py` file. The `pinionai` library will automatically import these functions so they can be called by the agent.

Note: A function in `/pinionai_extensions.py` file cannot be used, unless, you create a tool for the LLM to call it.

Dynamic Context with Variables

A powerful feature is the ability to embed agent variables directly into your function declarations using the `{var["variable_name"]}` syntax. When the LLM considers using your tool, these placeholders are dynamically replaced with the current values from the agent's session. This gives the LLM crucial context about what values to use for the function's parameters.

Please note with `{var['variable']}`, you can easily pass values to functions in `pinionai_extensions.py`

1. Within PinionAI Studio, navigate to the **Tools** page from the sidebar.
2. Ensure an account and agent are selected.
3. Expand the "New Tool" section.
4. Fill out the form with the following details:
 - **Tool Name:** A tool named `get_stock_data` to match the function name in `pinionai_extensions.py` file.
 - **Status:** A toggle to enable or disable the tool.
 - **Agent(s):** A multi-select field to associate the tool with one or more AI Agents. Only associated agents can use this tool.
 - **Functional Declaration JSON:** A text area for defining your own custom functions using an OpenAPI-compliant JSON schema. (see below) This tells the AI model what your function does, what parameters it accepts, and what it returns. This field should contain a JSON array of objects, where each object is a function declaration.
5. Click the "Create Tool" button to save the new tool.

Corresponding Stock Data Functional Declaration Please note the use of variables to pass the data. The **description** fields for its parameters tell the LLM to use the values from the agent's **stock_symbol** and **alphavantage_key** variables.

```
{
  "name": "get_stock_data",
  "description": "Fetches stock data from the Alpha Vantage API. Use this
to get stock prices, company overviews, and other financial data.",
  "parameters": {
    "type": "object",
    "properties": {
      "stock_lookup_function": {
        "type": "string",
        "description": "The specific Alpha Vantage function to call. We
will use the following function {var['stock_lookup_function']}"
      },
      "stock_symbol": {
        "type": "string",
        "description": "The stock ticker symbol for the company. we will
use {var['stock_symbol']}"
      },
      "alphavantage_key": {
        "type": "string",
        "description": "The API key required for authenticating with the
Alpha Vantage service. We will use {var['alphavantage_key']}"
      },
      "interval": {
        "type": "string",
        "description": "The time interval for time series data, e.g.,
'5min', '15min', '60min'. Required for functions like
'TIME_SERIES_INTRADAY'."
      }
    },
    "required": ["stock_lookup_function", "stock_symbol",
"alphavantage_key"]
  }
}
```

Pro Tip: Collecting inputs in the Intent.

Remember, you need to make sure you have Intents such as **check share price** (information intent) and **stock ticker symbol** and **stock research type** (inputs) which will collect and set the **stock_symbol** and **stock_lookup_functions** variables.

The **check share price** intent should include a completed **Required Variable Inputs (JSON)** field like the following

```
{
  "0": { "stock_symbol": "What is the stock ticker to you want to check" },
```

```
"1": {  
  "stock_lookup_function": "What do you want to check. You can say  
GLOBAL_QUOTE for current price information, TIME_SERIES_INTRADAY for  
today's price action, or TIME_SERIES_DAILY for recent price moves."  
}
```

Also, Tools are called in an LLM from a prompt. So you will need a prompt that 'uses' the Tool. This prompt example below instructs the model to call the custom function tool, (`get_stock_data`) to retrieve live data before generating a response.

- **Prompt Name:** `stock price – final prompt`
- **Type:** `final prompt`
- **Provider:** `google`
- **Model:** `gemini-2.0-flash`
- **Tools:** `get stock data`
- **Response Variable:** `stock_analysis_response`
- **Body:**

Retrieve stock information, and format it in markdown to make it easy to read. User question: {user_input}

To get the data, it will call the `get_stock_data` tool and custom python function. The stock ticker to gather information on is: {var["stock_symbol"]}. The function to perform for the lookup also uses: {var["stock_lookup_function"]} and the API key to use is: {var["alphavantage_key"]}